[Dung Vu] Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

- **Section**: Embedded Linux
- **Topic**: Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

---

1. ARM Boot Process

> Before an OS is installed and running, how does a machine boot and load that OS?

That question comes to a topic of Boot process

0. **Power-On Reset (Hardware)**

- Voltage stabilizes and triggers a hardware Reset signal.
- The CPU sets its **Program Counter** to a hardcoded memory address called the **Reset Vector**

> **Program Counter** (commonly called **Instruction Pointer** in x86-64): A CPU register that stores the memory address of the next instruction to be fetched and executed.

> **Reset Vector**: A hardcoded memory address pointing to the initial startup code. Stored in non-volatile memory such as ROM, NOR Flash, or EEPROM. Is fixed in hardware by design
> Intel x86 (Real Mode): `0xFFFF0`

> ARM Cortex-M: `0x00000004`

ARM uses a Multi-Stage Bootloader architecture, split into distinct Bootloader Stages (BL):

1. **BootROM**
   - Executes from on-chip ROM inside the SoC (the code at the **reset vector**)
   - Written by the vendor, so it cannot be modified.
   - Reads the boot source (SD/eMMC/NAND/SPI flash, selected by boot pins or fuses) and loads the SPL into internal SRAM.
2. **Secondary Program Loader (SPL)**
   - Executes from internal SRAM, because the full bootloader is too big to fit in Internal SRAM
   - Initialize DDR Controller => external DRAM becomes usable
   - Loads U-Boot into DRAM.
3. **Main Bootloader (U-Boot)**
   - Executes from external DRAM
   - Sets up the environment needed for the OS
   - Loads the **Linux Kernel** and **Device Tree** into RAM
   - Hands over control to the kerne
4. **Linux Kernel + Device Tree**
   - The kernel initializes memory management, drivers, and CPU scheduling
   - It spawns the first user-space process (`init`/`systemd`, PID 1)

2. ROM code / BootROM

ROM code:

- The first code a CPU executes after reset
- Is burned into the silicon at manufacturing => Can't be erased or reprogrammed

What it does:

- Minimal CPU and SoC setup
- Choose a boot source (SD, eMMC, NAND, SPI NOR, USB, UART)
- Load the SPL from that device into on-chip SRAM
- Verify the image's signature (if secure boot is enabled)
- Hand control to the SPL

3. Bootloader

A small program that is responsible for system preparation and OS loading.

The bootloader is stored in non-volatile memory that the BootROM can read. e.g. SD, eMMC, NAND, SPI flash, external source (USB, UART, network)

Some popular bootloaders:

- U-Boot (Das U-Boot): The de facto standard for embedded Linux.
- GRUB 2: The default on most Linux distributions on desktops. Supports BIOS and UEFI, many filesystems, and multiboot menus.
- Windows Boot Manager (bootmgr): Used by Windows

Revision #2
Created 2026-10-23 09:40:02 UTC by Dung Vu
Updated 2026-10-23 09:56:32 UTC by Dung Vu